

```

theory Week_3_sol
imports Main
begin

```

King's College London
Department of Informatics

Semester 1

Formal Verification: 6CCS3VER

Exercise Sheet 8

Exercise 8.1 Fold function

The fold function is a very generic function, that can be used to express multiple other interesting functions over lists.

Have a look at Isabelle/HOL's standard function *fold*, defined as follows:

```

fun fold:: "('a ⇒ 'b ⇒ 'b) ⇒ 'a list ⇒ 'b" where
  "fold f [] acc = acc"
| "fold f (x # xs) acc = fold f xs (f x acc)"

```

Write a function to compute the sum of the elements of a list. Define two versions, one direct recursive definition, and one using fold. Show that both are equal.

Hint: use automation!

```

fun list_sum :: "nat list ⇒ nat"

```

```

where
  "list_sum [] = 0"
| "list_sum (x # xs) = x + list_sum xs"

```

```

definition list_sum' :: "nat list ⇒ nat"

```

```

where "list_sum' xs ≡ fold (+) xs 0"

```

These are three equivalent proofs of the same lemma. The first one only uses substitution in an apply script, the second uses Isar equational reasoning, and the third uses automation. The second one looks like what you would write on pen-and-paper.

lemma “ $\bigwedge acc. acc + list_sum\ xs = fold\ (+)\ xs\ acc$ ”

proof(*induction xs*)

case *Nil*

thm *list_sum.simps(1)*

thm *fold.simps(1)*

show *?case*

apply(*subst list_sum.simps(1)*)

apply(*subst fold.simps(1)*)

apply(*subst add_0_right*)

 ..

next

case (*Cons a xs*)

show *?case*

apply(*subst list_sum.simps(2)*)

apply(*subst fold.simps(2)*)

apply(*subst add.assoc[symmetric]*)

apply(*subst add.commute*)

apply(*subst Cons.IH*)

 ..

qed

lemma “ $\bigwedge acc. acc + list_sum\ xs = fold\ (+)\ xs\ acc$ ”

proof(*induction xs*)

case *Nil*

have “ $acc + list_sum\ [] = acc + 0$ ”

apply(*subst list_sum.simps(1)*)

 ..

also have “ $\dots = acc$ ”

apply(*subst add_0_right*)

 ..

also have “ $\dots = fold\ (+)\ []\ acc$ ”

apply(*subst fold.simps(1)*)

 ..

finally show *?case*

 .

next

case (*Cons a xs*)

have “ $acc + list_sum\ (a\#\!xs) = acc + (a + list_sum\ xs)$ ”

apply(*subst list_sum.simps(2)*)

 ..

also have “ $\dots = acc + a + list_sum\ xs$ ”

apply(*subst add.assoc[symmetric]*)

 ..

also have “ $\dots = a + acc + list_sum\ xs$ ”

apply(*subst add.commute*)

 ..

also have “ $\dots = fold\ (+)\ xs\ (a+acc)$ ”

```

    apply(subst Cons.IH)
  ..
also have "... = fold (+) (a#xs) acc"
  apply(subst fold.simps(2)[symmetric])
  ..

finally show ?case
.
qed

lemma aux: " $\bigwedge a. \text{fold } (+) \text{ xs } a = \text{list\_sum xs} + a$ "
  by (induction xs) auto

lemma "list_sum xs = list_sum' xs"

  unfolding list_sum'_def by (simp add: aux)

```

Exercise 8.2 Tail recursive *reverse*

In Isabelle, there is the reverse function *rev*, which, given a list, computes another list with the same elements but in reverse order. Take a look into its definition:

```
thm rev.simps
```

Recall tail recursion: a function is tail recursive if in all recursive calls it does not call anything after itself¹. For instance, the implementation of *rev* in Isabelle is not tail recursive, because it calls *@* after it calls *rev* in the second equation.

Write an alternative implementation of the reverse function that is tail recursive.

Hint: recall that you can most of the times derive a tail recursive function using an accumulator argument.

```

fun rev':: "'a list  $\Rightarrow$  'a list  $\Rightarrow$  'a list" where
  "rev' [] acc = acc"
| "rev' (x #xs) acc = rev' xs (x # acc)"

```

```

fun rev_tr:: "'a list  $\Rightarrow$  'a list" where

```

```
  "rev_tr xs = rev' xs []"
```

Show that the two implementations are equivalent.

```

lemma rev'_append: "rev' xs ys = (rev' xs []) @ ys"
proof (induction xs arbitrary: ys)

```

¹See https://en.wikipedia.org/wiki/Tail_call. for a quick review.

```

    case Nil
    then show ?case
      by auto
next
case (Cons a xs)
have 1: "rev' xs (a # ys) = (rev' xs []) @ (a # ys)"
  apply(subst Cons)
..
have 2: "rev' xs [a] = (rev' xs []) @ [a]"
  apply(subst Cons)
..
show ?case
  using 1 2
  by simp
qed

```

lemma "rev_tr xs = rev xs"

```

proof(induction xs)
  case Nil
  then show ?case
    by auto
next
case (Cons a xs)
  then show ?case
    apply simp
    apply(subst rev'_append)
    by auto
qed

```

end